Commissioning Proxy (COMPRO) Test Setup & Troubleshooting Guide

Sep 24, 2026 · Kishok

This is the working reference for running the Matter TC-COMPRO-2.x test suite against `all-devices-app` (Commissioning Proxy) and `chip-lighting-app` (End Device), covering both BLE and WiFi-PAF transports.

Contents

Overview . . . 1
WiFi-PAF setup . . . 1
BLE setup . . . 2
Running each test . . . 4
Troubleshooting checklist . . . 5
Open issues . . . 6

Overview

TC-COMPRO-2.x validates the Matter **Commissioning Proxy** cluster: a Commissioner (TH) asks an already-commissioned Proxy device to discover and tunnel commissioning traffic to a nearby End Device (ED) over a transport the Commissioner can't reach directly.

Three roles, three separate concerns:

Role	Binary	What it needs
TH (test harness)	`TC_COMPRO_2_x.py`	Runs the Python test, drives the Proxy over IP
Proxy (DUT)	`all-devices-app --device commissioning-proxy:5`	BLE and/or WiFi-PAF radio, correctly commissioned onto the fabric
End Device (ED)	`chip-lighting-app`	Same transport(s) as the Proxy, discoverable by it

Two transports are exercised, controlled independently per test:

- **BLE** — proxy acts as a BLE central, connects to the ED's CHIPoBLE GATT service, tunnels PASE/commissioning over it.
- **WiFi-PAF** — proxy and ED use WiFi NAN (Neighbor Awareness Networking) publish/subscribe to discover each other, then tunnel over that link. Requires a NAN-USD-capable USB WiFi dongle on **both** machines — onboard RPi WiFi cannot do this.

WiFi-PAF setup

All of this is scripted — you shouldn't need to run these steps by hand.

1. One-time, per machine

```
./wifi_paf_env.sh initial_setup
```

Builds a patched, NAN-USD-capable `wpa_supplicant` (the patch is embedded in the script — nothing to download separately), installs it wherever it will actually get executed, and fixes the control-interface/D-Bus setup. Run this once on the Proxy and once on the End Device.

It will prompt to create `wpa_supplicant-def.conf` if missing — say yes. **That file should never contain a `network={}` block** — see Troubleshooting.

2. Every test session

```
./wifi_paf_env.sh renew_paf
```

Then launch each app directly (frequency must match on both sides):

```
# Proxy
./all-devices-app --device commissioning-proxy:5 --wifi --wifipaf "freq_list=2437" \
    --discriminator 3840
# End Device
./chip-lighting-app --wifi --wifipaf "freq_list=2437" --discriminator 3841
```

Confirm each side actually published: `grep -a "publish_id" <stdout log>` should show a number, not be empty.

If a dongle ever gets wedged (`firmware upload failed: -110` after a forced `rmmod`): physically unplug and replug it — a software-level reset usually isn't enough.

BLE setup

BLE uses the onboard radio — no dongle setup needed. But there's a known kernel/BlueZ bug that breaks BLE advertising entirely on newer kernels.

Known bug: BlueZ 5.72 advertising payload vs. kernel 6.8.0-1064+

Symptom:

```
FAIL: RegisterAdvertisement: GDBus.Error:org.bluez.Error.Failed: Failed to register
    advertisement
Disabling CHIPoBLE service due to error: ...BluezObjectManager.cpp:317
```

BlueZ 5.72 sends an advertising payload 8 bytes longer than it should be (a struct-size bug in `advertising.c`). Kernel `6.8.0-1047-raspi` silently tolerated this; `6.8.0-1064-raspi` added stricter validation and now rejects it outright, so the ED/Proxy can never register a BLE advertisement.

Fix — downgrade to kernel 6.8.0-1048-raspi:

```
sudo apt-get install -y linux-image-6.8.0-1048-raspi linux-modules-6.8.0-1048-raspi
sudo flash-kernel --force 6.8.0-1048-raspi
sudo cp /boot/vmlinuz-6.8.0-1048-raspi /boot/firmware/vmlinuz1048
sudo cp /boot/initrd.img-6.8.0-1048-raspi /boot/firmware/initrd1048
sudo cp --update=none /boot/firmware/config.txt /boot/firmware/config.txt.orig
sudo sed -i \
  -e 's/^kernel=vmlinuz$/kernel=vmlinuz1048/' \
```

```
  -e 's/^initramfs initrd\.img followkernel$/initramfs initrd1048 followkernel/' \
  /boot/firmware/config.txt
sudo reboot
```

Confirm after reboot: `uname -r` should show `6.8.0-1048-raspi`.

Apply this on both the Proxy and ED machines. A real fix is landing upstream in BlueZ for Noble (24.04) — check `apt list --upgradable | grep bluez` periodically so you can eventually move off the pinned kernel.

Recovering the BLE stack mid-session

If BLE has been through a lot of churn (repeated test runs, driver reloads elsewhere on the machine), a clean restart of just the Bluetooth stack is worth trying before deeper debugging:

```
sudo systemctl restart bluetooth
sudo hciconfig hci0 down
sudo hciconfig hci0 up
```

Switching between onboard and external Bluetooth

Some tests fail with the Raspberry Pi's built-in Bluetooth and need a USB Bluetooth adapter instead. `ble_select.sh` switches which one is powered on.

```
./ble_select.sh usb       # power off onboard, power on the USB adapter
./ble_select.sh onboard   # power off the USB adapter, power on onboard
```

The script finds each adapter by bus type (UART = onboard, USB = dongle), so it works even when `hci0`/`hci1` swap places between machines or reboots. If the requested adapter isn't present it falls back to the other one and says so.

When it finishes it prints the value to pass to the app, for example `--ble-controller 1`. Use the printed number every time — never assume which `hciN` is which.

```
./all-devices-app --device commissioning-proxy:5 --wifi --wifipaf "freq_list=2437" \
    --discriminator 3840 --ble-controller <N>
```

For the End Device, check whether your build supports the flag first: `./chip-lighting-app --help | grep ble-controller`.

Which tests need the external adapter

Test	Run on	Needs external BLE?	Why
TC-COMPRO-2.2 (dual-transport step)	Proxy	Yes	Scanning BLE and WiFi-PAF at once with the onboard chip found only one transport
TC-COMPRO-2.8 (combined-transport background scan)	Proxy	Yes	Same cause; combined scan returned no results on onboard
TC-COMPRO-2.4 (BLE connect)	Proxy	Not fixed by either	Onboard stalls ~15s; USB adapter aborts with HCI `0x3E` — see Open issues

Test	Run on	Needs external BLE?	Why
Single-transport tests	Either	No	BLE-only and WiFi-PAF-only tests passed on the onboard adapter

The external adapter only matters on the machine that does the scanning — the Proxy.

Running each test

Note: The commands and arguments below are examples prepared initially. For the most recent commands and arguments, refer to the **Python Script Validation Procedure** tab of the **Verification Steps Document**.

Commissioning flag — the one thing that varies most

Test	Flag	Why
2.2, 2.3, 2.7, 2.8	`--commissioning-method on-network`	Framework commissions the proxy before the test body runs
2.6	`--in-test-commissioning-method on-network`	Test needs a PASE-only window before commissioning (Phase 1), then commissions itself mid-test

Using the wrong one for 2.6 hangs forever re-resolving an uncommissioned node; using the wrong one for the others fails instantly at the framework's own commissioning step.

Standard BLE-only command (works for 2.2, 2.4, 2.6, 2.7, 2.8)

```
python3 TC_COMPRO_2_x.py \
    --commissioning-method on-network \
    --discriminator 3840 --passcode 20202021 \
    --storage-path admin_storage.json \
    --endpoint 5 \
    --string-arg ed_app_path:/path/to/chip-lighting-app \
    --string-arg ed_extra_args:--wifi \
    --string-arg ed_transport:ble \
    --int-arg ed_discriminator:3841 ed_passcode:20202021
```

WiFi-PAF command (needed for 2.3, and the dual-transport half of 2.2)

```
python3 TC_COMPRO_2_x.py \
    --commissioning-method on-network \
    --discriminator 3840 --passcode 20202021 \
    --storage-path admin_storage.json \
    --endpoint 5 \
    --string-arg ed_app_path:/path/to/chip-lighting-app \
    --string-arg 'ed_extra_args:--wifi --wifipaf freq_list=2437' \
    --string-arg ed_transport:wifipaf \
    --int-arg ed_discriminator:3841 ed_passcode:20202021
```

Quoting matters: if driving this through a JSON-based harness config (`test_parameters.string-arg`), the `ed_extra_args` value must stay quoted as one token, or the harness's space-delimited parser splits `--wifipaf` off as an unrecognized top-level argument.

Manual mode (no `ed_app_path`) — for debugging ED-side startup

Omit `ed_app_path`/`ed_extra_args`/`ed_serial_port` entirely; the test then pauses and prompts you to start/stop the ED by hand at each relevant step:

```
--string-arg ed_transport:wifipaf
--int-arg ed_discriminator:3841 ed_passcode:20202021
```

Use this when you need to see the ED's own stdout directly (the automated fixture sends it to `/dev/null` by default):

```
./chip-lighting-app --wifi --wifipaf "freq_list=2437" --discriminator 3841 > \
    /tmp/ed_stdout.log 2>&1 &
```

Troubleshooting checklist

Symptom	Likely cause	Fix
Test hangs forever "Re-trying resolve"	Wrong commissioning flag for this test	Check the flag table above
`Expected NumberOfResults == 0` but found 1+	Another team's rig advertising nearby (shared lab)	Confirm via discriminator/VendorID in the result; coordinate on scheduling, or physically isolate
`nansubscribe failed: ...InvalidArgs` + "Check wpa_supplicant supports discovery_only flag"	Wrong `wpa_supplicant` binary executing (see WiFi-PAF setup, step 2)	Verify with `strings | grep discovery_only`; fix whichever path `which wpa_supplicant` resolves to
`wpa_supplicant: failed to create proxy ... Unit wpa_supplicant.service is masked`	Service was masked earlier and never unmasked	`sudo systemctl unmask wpa_supplicant`, then start your own instance so it claims the D-Bus name first
`wpa_cli -i <iface> ...` → "No such file or directory"	`wpa_supplicant` bound to the wrong interface, or `ctrl_interface` missing from its config	Check `/etc/wpa_supplicant/wpa_supplicant.conf` has `ctrl_interface=DIR=/run/wpa_supplicant`; restart bound to the correct interface
`busctl status fi.w1.wpa_supplicant1` shows `-i wlan0`	systemd's instance reclaimed the D-Bus name	Mask (or unmask + ensure yours starts first) as in WiFi-PAF setup step 3
Full 10s `ProxyScanRequest` finds nothing, ED confirmed running	ED's `wpa_supplicant` never actually started NAN (check its own stdout for `publish_id`), or wrong `wpa_supplicant` binary	Check ED stdout; re-verify binary path per step 2
`FAIL: RegisterAdvertisement: ...org.bluez.Error.Failed`	Kernel/BlueZ advertising bug (kernel 6.8.0-1064+)	Downgrade to `6.8.0-1048-raspi` — see BLE setup

Symptom	Likely cause	Fix
`ProxyConnectRequest` → generic `Failure (0x1)` after ~19-20s, BLE connect + GATT discovery visibly succeed in `btmon` but the BTP handshake stalls	Suspected 2.4GHz RF coexistence between onboard BT and an active WiFi-PAF USB dongle on the *same* machine — **not fully confirmed, see Open issues**	Try physically separating the dongle from the board; see Open issues section
`mt76x2u ... firmware upload failed: -110` after `rmmod/modprobe`	Dongle chip left in a bad state by a forced module unload	Physically unplug and replug the dongle — sysfs unbind/bind is often not enough
`apt install` stuck on "Waiting for cache lock"	`unattended-upgrades` running (often a kernel update)	Let it finish — check `ps aux` / its log for real progress before ever force-killing it; a killed kernel upgrade can break the system
Second `ProxyConnectRequest` in a loop fails, no hardware for that transport	`Transport` attribute reflects what's **compiled in**, not what hardware is currently present/plugged in	Expected if you've deliberately removed hardware for one transport; either restore the hardware or rebuild the proxy without that feature (`chip_device_config_enable_wifipaf=false`)

Open issues

Dual-transport / concurrent-scan RF timing (mostly resolved)

TC-COMPRO-2.2's dual-transport check and TC-COMPRO-2.8's combined-transport background scan would intermittently find only one of BLE/WiFi-PAF, never both, when the Proxy had to scan both simultaneously. Confirmed via `btmon` that both transports work perfectly in isolation — only concurrent scanning was affected.

Fix that resolved it: swap the Proxy's onboard Bluetooth for a USB Bluetooth adapter (fully separate radio, `hci0`/`hci1` — use `ble_select.sh usb`/`ble_select.sh onboard` to switch), and pass `--ble-controller <N>` to `all-devices-app` to point at it. Confirmed passing repeatedly on 2.2 and 2.8 after this change.

If it recurs: retry first (this always had some timing sensitivity), then check `--ble-controller` still points at the right adapter (`hciconfig -a`, bus type may not map to the same `hciN` number after a reboot).

TC-COMPRO-2.4: BLE central-connect stall / `0x3E` abort

Status: unresolved, chipset-pairing suspected.

`ProxyConnectRequest` over BLE intermittently fails after ~15-20s with generic `Failure (0x1)`. Two distinct signatures seen via `btmon`:

1. Connection establishes, GATT discovery starts, then a silent ~14s gap before timing out (earlier finding, cause not confirmed).
2. With the USB Bluetooth adapter as central: connection completes (`LE Enhanced Connection Complete: Success`), then the **local** controller aborts ~250ms later with HCI reason `0x3E` ("Connection Failed to be Established") — repeats on all retries. This is a documented interoperability quirk seen between certain Realtek-central + Broadcom-peripheral pairings, not clearly a bug in either device alone.

Next steps: retry (both signatures show some randomness); try `linux-firmware` update for the USB adapter; try swapping which machine's chipset acts as central (`ble_select.sh` on the *other* machine) to see if the pairing is what matters.